

Guide de déploiement — Kasa (Netlify)

Ce document est le seul nécessaire pour comprendre le contexte du projet et le déployer sur Netlify de façon autonome et sécurisée. Il reflète l'état réel du dépôt au moment de la rédaction — pas une procédure Netlify générique.

Statut : déploiement de production réalisé et validé le 2026-07-23 ; **CD GitHub↔Netlify active depuis le 2026-07-23** (chaque `git push` sur `main` redéploie automatiquement, vérifié indépendamment — voir §6).

Site live : <https://kasa-react-app-283.netlify.app> (admin : <https://app.netlify.com/projects/kasa-react-app-283>). Les commandes et points d'attention ci-dessous ont été mis à jour avec ce qui a réellement été exécuté et rencontré — voir §8 pour le détail des difficultés rencontrées.

1. Contexte de l'application

Kasa est un **POC (proof of concept) front-end** de type Airbnb : un site vitrine de location d'appartements entre particuliers, développé dans le cadre d'un exercice de formation (voir la section "Scénario/Missions" du [README](#)).

Points clés à comprendre avant de déployer :

- **100 % front-end statique.** Il n'y a ni serveur applicatif, ni base de données, ni API distante. Le "backend" est un unique fichier statique, `public/logements.json`, servi tel quel avec le reste du build. Voir [docs/TECHNICAL_DOCUMENTATION.md §5](#) pour le détail.
- **Aucune authentification, aucun compte utilisateur, aucune donnée personnelle réelle.** Les données affichées (logements, hôtes, photos) sont des données de démonstration fournies avec le projet.
- **Stack technique** : React 19, React Router v7, Create React App (`react-scripts` 5.0.1), Sass compilé en CSS statique. Détail complet dans [docs/TECHNICAL_DOCUMENTATION.md](#).
- Conséquence directe pour le déploiement : **aucune variable d'environnement ni secret n'est requis pour que l'application fonctionne** (voir §4). Le risque en cas d'incident de déploiement est donc purement cosmétique/fonctionnel (page cassée), jamais une fuite de données sensibles.

2. Prérequis / dépendances

Outils locaux (pour builder/tester avant de déployer)

Outil	Version testée dans ce projet	Où le vérifier
-------	-------------------------------	----------------

Node.js	Non figée dans le dépôt — ni champ <code>engines</code> dans <code>package.json</code> , ni fichier <code>.nvmrc</code> . La seule version documentée est celle du <code>Dockerfile</code> (<code>node:22-bookworm-slim</code> , soit Node 22), utilisée avec succès pour builder ce projet.	<code>node --version</code>
yarn	Gestionnaire de paquets unique du projet (<code>yarn.lock</code> est le seul lockfile versionné ; <code>package-lock.json</code> a été retiré du dépôt et est désormais ignoré, voir <code>.gitignore</code>). Utilisé par le <code>Dockerfile</code> et pour ce premier déploiement Netlify. N'utilisez pas <code>npm install/npm ci</code> .	<code>yarn --version</code>

[ATTENTION] Comme Netlify n'a aucune indication de version Node dans ce dépôt, il utilisera sa version par défaut (régulièrement mise à jour par Netlify), ce qui peut différer de ce qui a été testé localement. Voir §4 pour la recommandation de fixer `NODE_VERSION`.

Comptes nécessaires

- **GitHub** : accès en lecture au dépôt qui héberge le code. [ATTENTION] Le remote `origin` configuré localement pointe vers `https://github.com/Nathan-Simonnet/Kasa.git`, mais GitHub redirige désormais vers `https://github.com/Nathan-Simonnet/Kasa-React.git` (le dépôt a été renommé). Utiliser cette nouvelle URL pour connecter Netlify au dépôt.
- **Netlify** : un compte avec les droits de créer un site (ou d'accéder à l'équipe/site existant si un site Netlify a déjà été créé pour ce projet — ce dépôt ne contient aucune trace locale d'un site déjà lié : pas de dossier `.netlify/`).
- **Netlify CLI** (optionnel mais recommandé pour la procédure de déploiement manuel et le rollback, voir §5 et §7) :

```
bash npm install -g netlify-cli netlify --version
```

3. Environnements / infrastructure

- **Un seul environnement existe : la production.** Site Netlify créé le 2026-07-23 via CLI : nom `kasa-react-app-283`, équipe `Openclassrooms` (slug `nathan-simonnet`), URL `https://kasa-react-app-283.netlify.app`, site ID `50a176f7-981c-44e0-b140-560097d1b3d5`.
- **La continuité (CD) avec GitHub est active depuis le 2026-07-23** (liée manuellement via l'UI Netlify, Option A du §5 — l'autorisation de l'app GitHub de Netlify est une étape interactive qui a été

faite directement dans le dashboard). Chaque `git push` sur `main` redéploie désormais automatiquement. Confirmé via `netlify api listSiteDeploys` : les déploiements des commits `5c5f729` et `8e0689d` (poussés avant la liaison) ont bien été rebuild automatiquement après coup, `commit_ref/commit_url/committer` renseignés et pointant vers le bon commit GitHub, `state: "ready", error_message: null`.

- **Domaine** : aucun domaine personnalisé configuré. Le site est accessible via le sous-domaine par défaut `kasa-react-app-283.netlify.app`. Le nom a été auto-suffixé (`-283`) par Netlify car un nom plus court était probablement déjà pris par un autre compte — c'est normal et sans conséquence.
- **Hébergement statique uniquement** : aucune Netlify Function, aucun proxy d'API, aucune base de données côté Netlify n'est nécessaire ni configurée.

4. Configuration et accès

Fichier de configuration Netlify

Il n'existe toujours aucun `netlify.toml` dans ce dépôt — la commande de build et le dossier de publication doivent donc être saisis manuellement dans le dashboard Netlify si vous liez le dépôt via l'UI (Option A, §5), ou passés explicitement en CLI (`--dir=build`, Option B).

En revanche, `public/_redirects` existe désormais (ajouté le 2026-07-23, avant le premier déploiement) et contient :

```
/* /index.html 200
```

- **Pourquoi c'est indispensable** : sans cette règle, Netlify renvoie une **404 native** sur toute URL autre que `/` en cas d'accès direct ou de rafraîchissement (ex. `/a-propos`, `/location/c67ab8a7`), puisqu'aucun fichier physique ne correspond à ces chemins — seul `index.html` doit répondre, et c'est React Router qui gère le routing côté client ensuite. C'est exactement le rôle que joue `nginx.conf` (`try_files $uri $uri/ /index.html`) dans le déploiement Docker existant (voir [docs/TECHNICAL_DOCUMENTATION.md §7](#)).
- Ce fichier, placé dans `public/`, est copié tel quel dans `build/_redirects` par `react-scripts build` — aucune étape manuelle supplémentaire n'est nécessaire au moment du build.
- **Validé en conditions réelles** sur `https://kasa-react-app-283.netlify.app` : accès direct à `/a-propos` et `/location/c67ab8a7` fonctionnels, voir §6 et §8.
- Si ce fichier venait à être supprimé par erreur, la règle équivalente peut être recréeée manuellement dans le dashboard : *Site configuration* → *Build & deploy* → *Post processing* → *Redirects and rewrites* (`/*` → `/index.html`, statut `200`) — mais la version versionnée dans le dépôt reste la source de vérité à privilégier.

Variables d'environnement / secrets

Recherche effectuée dans le code (`grep -rn "process.env\" src`) : **aucune variable d'environnement n'est lue par l'application**, et aucun fichier `.env/.env.example` n'existe dans le

dépôt. Il n'y a donc **aucun secret applicatif à configurer** pour que le site fonctionne.

Seule variable recommandée côté build (pas un secret) :

- **NODE_VERSION** — définie à **22** sur le site live (contexte **all**), via **netlify env:set NODE_VERSION 22**, pour fixer la version de Node utilisée par le build Netlify et éviter une dérive par rapport à la version testée (voir §2). Valeur alignée sur celle du **Dockerfile**.

Règle générale : si une variable d'environnement ou un secret devait être ajouté à l'avenir (ex. clé d'API), il doit être saisi uniquement via *Site configuration* → *Environment variables* du dashboard Netlify (ou **netlify env:set** en CLI, jamais commité en clair), et jamais ajouté à ce document ni à un fichier versionné du dépôt.

5. Procédure de déploiement

Étape 0 — Vérifications locales avant tout déploiement

```
git pull origin main yarn install --frozen-lockfile yarn lint CI=true yarn test --watchAll=false yarn build
```

yarn build doit se terminer par **Compiled successfully**, sans erreur (des avertissements de dépréciation Sass/Node non bloquants sont normaux, voir [docs/TECHNICAL_DOCUMENTATION.md](#)). Le dossier **build/** généré est ce que Netlify doit publier.

Option A — Déploiement continu via l'UI Netlify (recommandé pour la production)

1. Dans le dashboard Netlify : **Add new site** → **Import an existing project** → **GitHub**, sélectionner le dépôt **Nathan-Simonnet/Kasa-React** (voir §2 pour la nouvelle URL).
2. Choisir la branche **main**.
3. Renseigner manuellement (aucun **netlify.toml** ne les fournit) :
 - **Build command** : **yarn build**
 - **Publish directory** : **build**
4. Ajouter la variable **NODE_VERSION** avant le premier déploiement (voir §4 pour le pourquoi). Deux façons d'y arriver, au choix :
 - **Pendant l'assistant de création du site** : l'écran qui demande *Build command/Publish directory* (étape 3 ci-dessus) affiche aussi une section **"Environment variables"** avec un bouton **"New variable"** (ou "Add environment variables") — y renseigner **Key** : **NODE_VERSION** / **Value** : **22**, avant de cliquer sur **Deploy site**.
 - **Après coup, si le site est déjà créé** : dans le dashboard du site → **Site configuration** (menu de gauche) → **Environment variables** → bouton **Add a variable** → **Add a single variable** → renseigner **Key** : **NODE_VERSION**, **Value** : **22**, laisser le scope par défaut ("Same value for all deploy contexts", suffisant ici puisqu'il n'y a qu'un seul environnement, voir §3) → **Create variable**.
 - [ATTENTION] Si le site était déjà déployé avant l'ajout de la variable, celle-ci ne s'applique **pas rétroactivement** : il faut redéclencher un déploiement (**Deploys** → **Trigger deploy** → **Deploy site**)

pour qu'elle soit prise en compte.

- Équivalent en CLI, une fois le site lié localement (`netlify link`) : `netlify env:set NODE_VERSION 22` — c'est exactement la commande utilisée pour le site actuel `kasa-react-app-283` (voir Option B et §8), où cette variable est donc déjà configurée.

5. Configurer la règle de redirection SPA (voir §4) avant de tester la navigation.

6. Lancer le déploiement (**Deploy site**). Chaque nouveau `git push` sur `main` redéploiera automatiquement.

Option B — Déploiement manuel via Netlify CLI (méthode utilisée pour le premier déploiement)

Commandes exactes utilisées pour créer et déployer le site `kasa-react-app-283` :

```
netlify login # ouvre le navigateur pour OAuth, à valider soi-même netlify status # c
onfirme le compte et récupère le slug d'équipe (ex. "nathan-simonnet") netlify sites:
create -a nathan-simonnet -n kasa-react-app # crée le site ET le lie au dossier coura
nt (.netlify/state.json) netlify env:set NODE_VERSION 22 # fixe la version Node (voir
§4) yarn build # régénère build/ (inclut public/_redirects) netlify deploy --prod --
dir=build # déploiement de production
```

[ATTENTION] **Piège rencontré** : `netlify sites:create -a <slug>` attend le **slug** du compte/équipe, pas son nom affiché. `-a Openclassrooms` (nom affiché dans le dashboard) échoue avec **Error: createSiteInTeam error: 404: Not Found**. Le slug réel (`nathan-simonnet` dans ce cas) s'obtient via `netlify status` (champ `siteData/account`) ou `netlify api listAccountsForUser` (champ `slug`). Voir §8.

Si un site existe déjà et que vous voulez seulement le relier à un nouveau clone du dépôt : `netlify link` (interactif) à la place de `netlify sites:create`.

6. Validation post-déploiement

Vérifications manuelles dans le navigateur

Une fois le déploiement terminé (statut **Published** dans l'onglet *Deploys* du dashboard, ou fin de commande `netlify deploy --prod`), vérifier dans cet ordre. **Ces 6 points ont été exécutés et validés avec succès sur `https://kasa-react-app-283.netlify.app`, à la fois lors du premier déploiement manuel (2026-07-23) et après activation de la CD GitHub (2026-07-23)** :

1. **Log de build** : dans l'onglet *Deploys* → [dernier déploiement] → *Deploy log*, confirmer la présence de `Compiled successfully`, et l'absence d'erreur (des warnings Sass/Node sont attendus, voir §5). [OK]
2. **Page d'accueil** (`https://<nom-du-site>.netlify.app/`) : les cartes de logements doivent s'afficher (preuve que `logements.json` est bien servi). [OK] Les 20 logements s'affichent.
3. **Navigation SPA** : cliquer sur un logement → l'URL doit passer à `/location/<id>` et afficher le détail (carrousel, notation, description). [OK]

4. **Rafraîchissement direct d'une route profonde** : recharger la page sur `/a-propos` ou `/location/<id>` (F5) → doit afficher la page correspondante, **pas une 404 Netlify**. [OK] Testé sur `/a-propos` et `/location/c67ab8a7` en accès direct (pas juste en navigation cliquée) — grâce à `public/_redirects` (voir §4). Si une 404 Netlify apparaît malgré tout, ce fichier a probablement été retiré ou mal déployé.
5. **Gestion d'erreur applicative** : naviguer vers `/location/id-inexistant` → doit afficher la page 404 de l'application (`Oups! La page que vous demandez n'existe pas.`), pas une erreur Netlify. [OK]
6. **Console navigateur** : aucune erreur. [OK] Aucun message d'erreur ni warning sur les 4 pages testées.

Une vérification au navigateur est utile mais reste **manuelle et non reproductible automatiquement** — les méthodes suivantes permettent de confirmer la même chose sans réouvrir un navigateur à chaque déploiement, et de vérifier des points qu'un simple coup d'œil ne révèle pas (origine réelle du déploiement, valeur exacte des variables d'environnement).

Tests end-to-end Playwright contre le site déployé

`playwright.config.js` accepte une variable `PLAYWRIGHT_BASE_URL` : si elle est définie, la suite s'exécute contre cette URL au lieu de démarrer un serveur de dev local — pratique pour rejouer automatiquement les mêmes scénarios que la validation manuelle (accueil → détail, ID inconnu → 404, navigation "À propos") contre le site réellement déployé :

```
PLAYWRIGHT_BASE_URL=https://kasa-react-app-283.netlify.app yarn test:e2e
```

[OK] Exécuté le 2026-07-23 contre le site live : 3/3 tests passés (`la page d'accueil affiche des logements et mène à une page de détail, un identifiant de logement inconnu redirige vers la page 404, la navigation vers "À propos" fonctionne sans rechargement complet`). Le comportement par défaut sans cette variable (serveur local) reste inchangé et a été revérifié en parallèle.

Confirmer que le déploiement vient bien de la CD GitHub (pas d'un reliquat manuel)

Un déploiement "Published" ne prouve pas à lui seul qu'il a été déclenché par le bon mécanisme. Depuis le CLI (site déjà lié via `.netlify/state.json`, voir §5 Option B) :

```
netlify api listSiteDeploys --data '{"site_id":"<SITE_ID>"}
```

Dans la réponse, pour le déploiement le plus récent, vérifier :

- `"context": "production"` et `"state": "ready"`
- `"commit_ref"` renseigné avec le SHA du dernier commit poussé sur `main` (comparer avec `git log -1 --format=%H`), et `"commit_url"` pointant vers `github.com/Nathan-Simonnet/Kasa-React/commit/...`
- `"error_message": null`

- Dans `"summary.messages"`, une entrée `"1 redirect rule processed" / "All redirect rules deployed without errors."` — confirme que `public/_redirects` (§4) est bien pris en compte à chaque build, pas seulement lors du premier déploiement manuel.

[OK] Vérifié le 2026-07-23 : les déploiements des commits `5c5f729` et `8e0689d` (poussés juste avant la liaison CD) ont été rebuild automatiquement après la liaison, avec ces informations correctement renseignées.

Confirmer la valeur réelle des variables d'environnement

Une variable ajoutée via l'UI peut contenir une faute de frappe invisible à l'œil (espace, mauvaise clé). Vérifier la valeur telle qu'enregistrée côté Netlify plutôt que de se fier à ce qui a été saisi :

```
netlify env:list --json
```

[OK] Vérifié le 2026-07-23 : `{"NODE_VERSION": "22"}`, conforme à ce qui est recommandé au §4.

7. Rollback / gestion des incidents

Netlify conserve l'historique de tous les déploiements précédents, ce qui permet un rollback quasi instantané **sans rebuild ni revert git**.

Rollback via le dashboard (méthode recommandée, la plus fiable)

1. Aller dans l'onglet **Deploys** du site.
2. Repérer le dernier déploiement qui fonctionnait correctement (statut *Published* précédent).
3. Cliquer sur ce déploiement puis sur **Publish deploy** — il redevient immédiatement la version servie en production, sans toucher au code du dépôt.

Rollback via Netlify CLI / API (alternative scriptable)

```
netlify api listSiteDeploys --data '{"site_id":"<SITE_ID>"}
```

récupère la liste des déploiements et leurs `deploy_id`, puis :

```
netlify api restoreSiteDeploy --data '{"site_id":"<SITE_ID>","deploy_id":"<DEPLOY_ID_A_RESTAURER>"}
```

restaure ce déploiement comme version publiée (`<SITE_ID>` visible dans *Site configuration* → *General* → *Site details*, ou directement `50a176f7-981c-44e0-b140-560097d1b3d5` pour le site actuel `kasa-react-app-283`). Ces deux commandes n'ont pas encore été nécessaires en pratique (aucun incident depuis le premier déploiement) mais ont été confirmées comme des méthodes d'API valides (`netlify api <méthode>` accepte n'importe quelle méthode de l'API Netlify).

Marche à suivre en cas d'incident

1. **Rollback immédiat** vers le dernier déploiement sain (méthode ci-dessus) pour couper court à l'impact utilisateur — cette action ne dépend d'aucune correction de code.
2. **Diagnostiquer** ensuite à tête reposée :
 - Build qui échoue sur Netlify mais pas en local → comparer la version de Node utilisée (voir §4, `NODE_VERSION`) ; vérifier qu'aucun `package-lock.json` n'a été recommité par erreur (voir §2 — `yarn` est le seul gestionnaire de paquets du projet).
 - Page blanche ou assets manquants après déploiement → vérifier qu'aucun champ `homepage` n'a été ajouté à `package.json` (absent aujourd'hui, ce qui suppose un hébergement à la racine du domaine) et que le *Publish directory* est bien `build`.
 - 404 sur une route déjà fonctionnelle auparavant → la règle de redirection SPA (§4) a probablement été perdue lors d'une reconfiguration du site ; la restaurer.
3. **Corriger sur une branche**, valider en local (§5, Étape 0), rouvrir une pull request, puis ne redéploier en production qu'une fois la correction validée — ne jamais corriger un incident directement en modifiant les réglages du site sans trace dans le dépôt si le correctif touche au code.

8. Journal du premier déploiement (2026-07-23)

Section tenue à jour à chaque déploiement notable, pour que les difficultés déjà rencontrées ne soient pas redécouvertes à froid.

Étape	Résultat	Détail
<code>netlify login</code>	[OK] Sans difficulté	OAuth via navigateur, compte <code>nathan.simonnet@gmail.com</code> , équipe <code>Openclassrooms</code> .
<code>netlify sites:create -a Openclassrooms -n kasa-react-app</code>	[ECHEC] Échec (<code>404: Not Found</code>)	Voir §5 — <code>-a</code> attend le slug (<code>nathan-simonnet</code>), pas le nom affiché (<code>Openclassrooms</code>). Corrigé en récupérant le slug via <code>netlify status</code> .

<pre>netlify sites:create -a nathan-simonnet -n kasa-react-app</pre>	[ATTENTION] Nom auto-suffixé	Le nom demandé était probablement déjà pris ailleurs sur Netlify (les noms de site sont globaux, tous comptes confondus) ; Netlify a créé <code>kasa-react-app-283</code> sans erreur ni avertissement explicite — à vérifier dans la réponse JSON ("<code>name</code>") plutôt que de supposer que le nom demandé a été pris tel quel.
Ajout de <code>public/_redirects</code> avant le build	[OK] Nécessaire	Sans ce fichier, <code>/a-propos</code> et <code>/location/:id</code> auraient renvoyé une 404 Netlify au premier accès direct — anticipé dès la rédaction initiale de ce document (§4), confirmé nécessaire en pratique.
<pre>netlify env:set NODE_VERSION 22</pre>	[OK] Sans difficulté	Appliqué au contexte <code>all</code> du site.
<pre>yarn build puis netlify deploy --prod --dir=build</pre>	[OK] Sans erreur	<code>Compiled successfully.</code> , seuls les warnings de dépréciation Sass/Node habituels (voir docs/TECHNICAL_DOCUMENTATION.md).
Validation post-déploiement (§6)	[OK] 6/6	Aucune régression par rapport au comportement local.
Suppression de <code>package-lock.json</code> (post-déploiement)	[OK] Correction appliquée	Les deux lockfiles coexistaient encore au moment du premier déploiement, ce qui laissait le choix <code>npm run build</code> vs <code>yarn build</code> ambigu dans ce guide. Netlify auto-détecte le gestionnaire de paquets depuis le lockfile présent : avec les deux, le résultat n'était pas garanti stable. <code>package-lock.json</code> retiré, <code>yarn</code> fixé comme unique gestionnaire (voir §2). Vérifié : <code>yarn install --frozen-lockfile</code>, <code>yarn lint</code>, <code>yarn test</code>, <code>yarn build</code> passent tous après suppression.

Liaison CD GitHub↔Netlify (via l'UI, par l'utilisateur) + <code>NODE_VERSION</code>	[OK] Sans difficulté rapportée	Faite directement dans le dashboard (étape interactive, non réalisable en CLI). Vérifiée de façon indépendante à froid le 2026-07-23 (voir §6) : <code>netlify api listSiteDeploys</code> confirme que les commits <code>5c5f729</code> et <code>8e0689d</code> ont bien été rebuild automatiquement (<code>commit_ref/commit_url</code> corrects, <code>state: ready</code>, <code>error_message: null</code>, règle <code>_redirects</code> traitée sans erreur) ; <code>netlify env:list --json</code> confirme <code>NODE_VERSION=22</code> ; <code>PLAYWRIGHT_BASE_URL=...</code> <code>yarn test:e2e</code> : 3/3 tests passés contre le site live.
--	---------------------------------------	--

Non fait à ce stade, à prévoir séparément si besoin :

- Domaine personnalisé (aucun domaine n'existait à documenter, voir §3).
- Deploy Previews automatiques sur les pull requests (activés par défaut une fois la CD liée, mais pas encore testés avec une PR réelle).

Documents liés : [docs/TECHNICAL_DOCUMENTATION.md](#) (architecture, Docker/nginx, conventions), [docs/TESTING_STRATEGY.md](#) (tests à faire passer avant tout déploiement).